

ChatGPT DOM Structure Reference

Document Purpose

This is a technical reference documenting the actual DOM structure of ChatGPT conversations as of November 2025, extracted from live page inspection. Use this as the source of truth for implementing the export fix.

Message Container Structure

Root Element

All messages are wrapped in:

```
html

<article
  data-turn="user|assistant"
  data-turn-id="[uuid]"
  data-testid="conversation-turn-N"
  data-scroll-anchor="false">
```

Key selector: `article[data-turn]` or `article[data-testid^="conversation-turn"]`

Message Type 1: Simple User Message

Structure

```
html

<article data-turn="user" data-turn-id="a92ee933-c425-4e7b-8110-dd167700eae9">
  <div data-message-author-role="user"
    data-message-id="a92ee933-c425-4e7b-8110-dd167700eae9">
    <div class="user-message-bubble-color ...">
      <div class="whitespace-pre-wrap">
        [USER MESSAGE TEXT]
      </div>
    </div>
  </div>
</article>
```

Key Patterns

- **Role attribute:** `data-message-author-role="user"`
 - **Content location:** `.whitespace-pre-wrap` inside `.user-message-bubble-color`
 - **NO thinking labels** should ever appear on user messages
-

Message Type 2: Simple Assistant Message

Structure

```
html

<article data-turn="assistant" data-turn-id="4a6b9ab7-ad9e-49ed-b4bd-a72863ce89e8">
  <div data-message-author-role="assistant"
    data-message-id="c43cc75b-6761-414b-9ac6-5025c657aa86"
    data-message-model-slug="gpt-5">
    <div class="markdown prose ...">
      [ASSISTANT MESSAGE CONTENT]
    </div>
  </div>
</article>
```

Key Patterns

- **Role attribute:** `data-message-author-role="assistant"`
 - **Model info:** `data-message-model-slug="gpt-5"` (or other model)
 - **Content location:** `.markdown.prose` div
-

Message Type 3: Assistant with Thinking Label

Structure

html

```
<article data-turn="assistant" data-turn-id="2807ef64-4f8a-4250-b946-d5dde564568a">
  <!-- THINKING LABEL SECTION -->
  <div class="relative my-1 min-h-6">
    <span class="flex items-center gap-1 ... text-token-text-secondary ...">
      Thought for 18s
      <svg>...</svg>
    </span>
  </div>

  <!-- MESSAGE CONTENT SECTION -->
  <div data-message-author-role="assistant" data-message-id="...">
    <div class="markdown prose ...">
      [CONTENT]
    </div>
  </div>
</article>
```

Key Patterns

- **Thinking label location:** ABOVE the message content div, in a separate structural div
- **Thinking label text:** Inside `<span>` with classes including `text-token-text-secondary`
- **Pattern:** "Thought for [time]" or "Stopped thinking" or other states
- **Structural relationship:** Thinking div is SIBLING to message content div, not parent/child

CRITICAL: Thinking Label Selector Rules

javascript

```
// CORRECT: Look for thinking in structural position
const thinkingDiv = article.querySelector('relative.my-1.min-h-6');
const thinkingSpan = thinkingDiv?.querySelector('span.text-token-text-secondary');
const thinkingText = thinkingSpan?.textContent?.trim();

// WRONG: Search all text nodes (will match user content)
const allText = article.textContent; // DON'T DO THIS
```

Message Type 4: Multi-Stage Thinking

Structure

html

```
<article data-turn="assistant" data-turn-id="50ff8e39-d17c-4647-ae27-0eaa75eec68b">
  <!-- THINKING LABEL 1 -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for 5s</span>
  </div>

  <!-- THINKING LABEL 2 -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for 29s</span>
  </div>

  <!-- THINKING LABEL 3 -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for 9s</span>
  </div>

  <!-- THINKING LABEL 4 -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for a few seconds</span>
  </div>

  <!-- THINKING LABEL 5 -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for 35s</span>
  </div>

  <!-- ACTUAL MESSAGE CONTENT -->
  <div data-message-author-role="assistant" data-message-id="...">
    <div class="markdown prose ...">
      [CONTENT]
    </div>
  </div>
</article>
```

Key Patterns

- **Multiple thinking divs:** Each with same structure `div.relative.my-1.min-h-6`
 - **Sequence preserved:** Order of divs = chronological order of thinking stages
 - **All before content:** All thinking divs appear BEFORE the message content div
 - **Export requirement:** Capture ALL labels, preserve sequence
-

Message Type 5: Canvas Artifact (Document)

Structure

```
html

<article data-turn="assistant" data-turn-id="2807ef64-4f8a-4250-b946-d5dde564568a">
  <!-- THINKING LABEL -->
  <div class="relative my-1 min-h-6">
    <span class="...">Thought for 18s</span>
  </div>

  <!-- CANVAS DOCUMENT -->
  <div id="textdoc-message-69027c00a1c08191900ae470891a4712"
    class="popover bg-token-bg-primary ... rounded-3xl ...">

    <!-- CANVAS HEADER -->
    <div class="sticky top-0 ...">
      <div class="...">
        <span class="truncate text-token-text-primary font-semibold">
          Mosaic → Claude Code Prompt Framework
        </span>
      </div>
    </div>

    <!-- CANVAS CONTENT (ProseMirror) -->
    <div class="_main_5jn6z_1 markdown prose ... ProseMirror"
      contenteditable="false">
      <h1><span>Mosaic → Claude Code Prompt Framework...</span></h1>
      <blockquote><p><span>Purpose: a reusable...</span></p></blockquote>
      [RICH DOCUMENT CONTENT]
    </div>
  </div>
</article>
```

Key Patterns

- **ID pattern:** `id^="textdoc-message-"` (starts with "textdoc-message-")
- **Class:** `popover` with `rounded-3xl`
- **Title location:** `.truncate.text-token-text-primary.font-semibold` in header
- **Content location:** `._main_5jn6z_1.markdown.prose.ProseMirror` or similar ProseMirror classes
- **Thinking before canvas:** Thinking label appears before the canvas div
- **Export requirement:** Mark as canvas artifact, preserve document structure

Canvas Detection Logic

javascript

```
function isCanvasArtifact(element) {  
  // Method 1: Check for textdoc ID  
  const hasTextdocId = element.id?.startsWith('textdoc-message-');  
  
  // Method 2: Check for canvas structure  
  const hasPopoverClass = element.classList.contains('popover');  
  const hasProseMirror = element.querySelector('.ProseMirror') !== null;  
  
  return hasTextdocId || (hasPopoverClass && hasProseMirror);  
}
```

Message Type 6: Stopped Thinking (Empty Response)

Structure

html

```
<article data-turn="assistant" data-turn-id="1d839eb6-48de-4726-93ff-f371f9e1451b">  
  <div class="relative my-1 min-h-6">  
    <span class="...">Stopped thinking<svg>...</svg></span>  
  </div>  
  <!-- NO MESSAGE CONTENT DIV -->  
</article>
```

Key Patterns

- **Has thinking label:** "Stopped thinking" or similar error state
- **NO content div:** No `data-message-author-role` div with actual content
- **Role:** Still an assistant message (determined by article data-turn)
- **Export requirement:** Create message with thinking label but empty content, mark as incomplete/error

Message Type 7: User Message with File Attachment

Structure

html

```
<article data-turn="user" data-turn-id="55f0e7f3-ecb2-466a-99d2-4f317a0ef989">
  <div data-message-author-role="user" data-message-id="...">

    <!-- FILE ATTACHMENT PREVIEW -->
    <div class="flex gap-2 flex-wrap ...">
      <div class="group ... relative inline-block ...">
        <div class="border-token-border-default ... border rounded-xl">
          <div class="flex flex-row items-center gap-2">
            <div class="... rounded-lg" style="background-color: rgb(146, 79, 247);">
              <svg>...</svg> <!-- File type icon -->
            </div>
            <div class="overflow-hidden">
              <div class="truncate font-semibold">Mosaic.zip</div>
              <div class="text-token-text-secondary truncate">Zip Archive</div>
            </div>
          </div>
        </div>
      </div>
    </div>
    </div>
    </div>
    </div>

    <!-- USER MESSAGE TEXT -->
    <div class="user-message-bubble-color ...">
      <div class="whitespace-pre-wrap">
        Here is the codebase to fully unpack...
      </div>
    </div>
  </div>
</article>
```

Key Patterns

- **File preview:** Appears BEFORE the text bubble
- **File name:** `.truncate.font-semibold` contains filename
- **File type:** `.text-token-text-secondary.truncate` contains type description
- **Border:** `.border-token-border-default.border.rounded-xl` wraps file preview
- **Export requirement:** Extract filename and type, mark message as having attachment

File Attachment Detection Logic

javascript

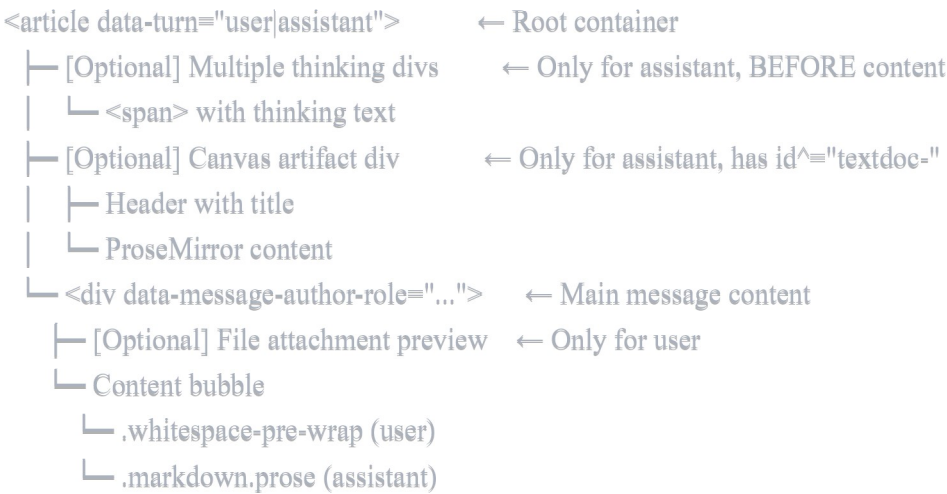
```
function detectFileAttachment(userMessageDiv) {
  const filePreview = userMessageDiv.querySelector('.border-token-border-default.border.rounded-xl');
  if (!filePreview) return null;

  const fileName = filePreview.querySelector('.truncate.font-semibold')?.textContent?.trim();
  const fileType = filePreview.querySelector('.text-token-text-secondary.truncate')?.textContent?.trim();

  if (fileName && fileType) {
    return { fileName, fileType };
  }

  return null;
}
```

Structural Hierarchy Summary



Critical Rules for Export Logic

Rule 1: Role Detection

javascript

```
// ALWAYS check article data-turn attribute FIRST
const role = article.getAttribute('data-turn'); // 'user' or 'assistant'

// Fallback to data-message-author-role on child div
if (!role) {
  const msgDiv = article.querySelector('[data-message-author-role]');
  role = msgDiv?.getAttribute('data-message-author-role');
}

// NEVER infer role from thinking labels
```

Rule 2: Thinking Label Detection

javascript

```
// ONLY look for thinking labels if role === 'assistant'
if (role !== 'assistant') {
  // Skip thinking detection entirely for user messages
  return { labels: [], expandable: false };
}

// ONLY search in structural thinking divs, not all text
const thinkingDivs = article.querySelectorAll('.relative.my-1.min-h-6');
const labels = [];

thinkingDivs.forEach(div => {
  const span = div.querySelector('span.text-token-text-secondary');
  const text = span?.textContent?.trim();

  if (text && /Thought for|Stopped|Analyzing|Processing/i.test(text)) {
    labels.push({ text, element: span });
  }
});
```

Rule 3: Canvas Detection

javascript

```
// Check for canvas BEFORE regular content extraction
const canvasDiv = article.querySelector('[id^="textdoc-message-"]');
if (canvasDiv) {
  const title = canvasDiv.querySelector('.truncate.text-token-text-primary.font-semibold')?.textContent;
  const content = canvasDiv.querySelector('.ProseMirror');

  return {
    isCanvas: true,
    title: title,
    content: content // Extract separately
  };
}
```

Rule 4: Multi-Stage Thinking

javascript

```
// Preserve ALL thinking labels in order
const thinkingLabels = []; // Preserves sequence

article.querySelectorAll('.relative.my-1.min-h-6').forEach(div => {
  const span = div.querySelector('span');
  if (span && /Thought for/i.test(span.textContent)) {
    thinkingLabels.push({
      text: span.textContent.trim(),
      order: thinkingLabels.length
    });
  }
});

// Don't collapse to single label - keep array
return { thinkingSequence: thinkingLabels };
```

Rule 5: Empty Assistant Messages

javascript

```
// Message with thinking but no content = valid message
const hasThinking = thinkingLabels.length > 0;
const hasContent = article.querySelector('[data-message-author-role] .markdown.prose');

if (hasThinking && !hasContent) {
  return {
    role: 'assistant',
    thinking: thinkingLabels,
    content: "",
    incomplete: /Stopped|Failed|Error/i.test(thinkingLabels[0]?.text)
  };
}
```

CSS Selectors Reference

Finding Messages

javascript

```
// Primary: Articles with data-turn
document.querySelectorAll('article[data-turn]')

// Secondary: Conversation turns
document.querySelectorAll('[data-testid^="conversation-turn"]')

// Tertiary: Message role divs
document.querySelectorAll('[data-message-author-role]')
```

Role Detection

javascript

```
// Article level
article.getAttribute('data-turn') // 'user' or 'assistant'

// Message div level
div.getAttribute('data-message-author-role') // 'user' or 'assistant'

// Model info (assistant only)
div.getAttribute('data-message-model-slug') // 'gpt-5', 'o1-mini', etc.
```

Thinking Labels

```
javascript
```

```
// Thinking container divs (multiple possible)
```

```
article.querySelector('.relative.my-1.min-h-6')
```

```
// Thinking text
```

```
thinkingDiv.querySelector('span.text-token-text-secondary')
```

```
// Expandable button (if present)
```

```
thinkingDiv.querySelector('button[aria-expanded]')
```

Content Extraction

```
javascript
```

```
// User message content
```

```
article.querySelector('.user-message-bubble-color .whitespace-pre-wrap')
```

```
// Assistant message content
```

```
article.querySelector('[data-message-author-role="assistant"] .markdown.prose')
```

```
// Canvas content
```

```
article.querySelector('[id^="textdoc-message-"] .ProseMirror')
```

File Attachments (User only)

```
javascript
```

```
// File preview container
```

```
article.querySelector('.border-token-border-default.border.rounded-xl')
```

```
// File name
```

```
filePreview.querySelector('.truncate.font-semibold')
```

```
// File type
```

```
filePreview.querySelector('.text-token-text-secondary.truncate')
```

Testing Checklist

Use these test cases to validate your extraction logic:

- ☐ Simple user message → role='user', no thinking
 - ☐ Simple assistant message → role='assistant', no thinking
 - ☐ Assistant with "Thought for X" → thinking label extracted
 - ☐ Assistant with 5 thinking stages → all 5 labels in sequence
 - ☐ Canvas document → isCanvas=true, title extracted
 - ☐ "Stopped thinking" with no content → empty assistant message, incomplete=true
 - ☐ User with file attachment → attachment metadata extracted
 - ☐ User message containing text "I thought for a while" → NO thinking label (it's content, not a label)
 - ☐ Mixed conversation → correct role for each message
-

Common Pitfalls to Avoid

✗ DON'T: Search all text nodes

```
javascript

// WRONG: Will match "Thought for" in user message content
const allText = container.innerText;
if (/Thought for/.test(allText)) { ... }
```

✓ DO: Search specific structural locations

```
javascript

// CORRECT: Only look in thinking label divs
const thinkingDivs = container.querySelectorAll('.relative.my-1.min-h-6');
```

✗ DON'T: Use thinking to determine role

```
javascript

// WRONG: Backward logic
if (hasThinking) {
  role = 'assistant'; // Could attach thinking to user message
}
```

✓ DO: Determine role first, then check thinking

```
javascript

// CORRECT: Role from structure, thinking only if assistant
const role = article.getAttribute('data-turn');
if (role === 'assistant') {
  const thinking = detectThinking(article);
}
```

✗ DON'T: Collapse thinking sequences

```
javascript
```

```
// WRONG: Loses multi-stage information
```

```
const thinking = thinkingLabels.length > 0 ? thinkingLabels[0] : null;
```

✓ DO: Preserve all thinking labels

```
javascript
```

```
// CORRECT: Keep sequence
```

```
const thinking = {  
  labels: thinkingLabels, // Array of all labels  
  sequence: thinkingLabels.map((l, i) => ({ ...l, order: i })))  
};
```

Version Info

- **Document Created:** 2025-11-03
- **ChatGPT Version:** Current production UI
- **Models Tested:** gpt-5-thinking, gpt-5, o1-mini
- **Browser:** Firefox (but structure should be consistent)

Updates Required

If ChatGPT's DOM structure changes, update this document with:

1. New CSS class patterns
2. Changed attribute names
3. Different structural relationships
4. Screenshots of new structure

Keep this document as the single source of truth for export logic.